Projektuppgift

DT211G – Frontend-baserad webbutveckling

Projektuppgift
Webbapplikation/Mashup

Hanna Lindkvist

Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Hanna Lindkvist, hali2507@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

Sammanfattning

Projektuppgiften har som syfte att utveckla en webbapplikation så kallad ”mashup” som kombinerar minst två olika JSON-baserade webbtjänster (APIer). Användargränssnittet ska vara dynamiskt vilket innebär att innehållet ska uppdateras med JavaScript och AJAX-anrop utan att sidan laddas om.

Projektuppgiften ska utföras utan frontend-ramverk och ha en automatiserad arbetsprocess baserad på node.js för att optimera, komprimera och publicera webbapplikationen.

Projektet har mynnat ut i en fungera ensides webbapplikation där majoriteten av innehållet uppdateras med JavaScript och AJAX-anrop.

Nyckelord: Frontend-baserad webbutveckling, JavaScript, Automatiserad arbetsprocess, AJAX-anrop, SASS/SCSS, node.js, Vite.

Innehåll

Sammanfattning iii

Projektuppgiften har som syfte att utveckla en webbapplikation så kallad ”mashup” som kombinerar minst två olika JSON-baserade webbtjänster (APIer). Användargränssnittet ska vara dynamiskt vilket innebär att innehållet ska uppdateras med JavaScript och AJAX-anrop utan att sidan laddas om. iii

Projektuppgiften ska utföras utan frontend-ramverk och ha en automatiserad arbetsprocess baserad på node.js för att optimera, komprimera och publicera webbapplikationen. iii

Projektet har mynnat ut i en fungera ensides webbapplikation där majoriteten av innehållet uppdateras med JavaScript och AJAX-anrop. iii

Nyckelord: Frontend-baserad webbutveckling, JavaScript, Automatierad arbetsprocess, AJAX-anrop, SASS/SCSS, node.js, Vite iii

1 Introduktion 1

1.1 Målgruppsanalys 1

2 Teori 2

2.1 Automatiserad arbetsprocess 2

2.2 CSS Animationer 2

2.3 CSS-preprocessorer 3

2.4 JavaScript 3

2.5 JSDoc 4

3 Metod 5

4 Konstruktion 6

5 Resultat 10

6 Slutsatser 11

1 Introduktion

I projektet ska en webbapplikation skapas vars syfte är att visa Stockholm Stads badplatser, specifik information om respektive badplats, en karta med markering för badplatsens adress och en väderprognos för det området.

Innehållet till webbapplikationen ska hämtas från Stockholms Stads Öppna Datas [19] webbtjänst Badplats (ur Hitta Service API) för informationer om badplatserna och som ska kombineras med Nominatim (OpenStreetMap) [20] för GPS-koordinater för att skapa en ut markerad karta samt Meteoblue [21] för väderprognos.

Webbapplikationen ska vara en webbsida där innehållet uppdateras utan att sidan laddar om. Först ska alla badplatser i Stockholm Stad listas och möjlighet att söka i listan ska finnas. En animation ska glida över toppen på skärmen när man kommer in på webbsidan för att ta fokus från eventuell fördröjning av hämtning från API. Varje badplats i listan ska zoomas när muspekare förs över för att tydligt visa vilket alternativ som är i fokus. Vid klick på en badplats ska specifik utökad information visas om badplatsen samt att en karta ska visa vart badplatsen ligger och en väderprognos för platsen ska visas.

Webbapplikationen ska ha en genomgående grafisk profil och vara fullt responsiv och fungera oavsett enhet eller skärmstorlek. Färgschemat ska anpassas utefter användares val av mörkt eller ljust tema. För att underhållet för webbapplikationen ska bli smidigt kommer SASS användas för CSS-styling där variabler, mixin, nesting och partials kommer bidra till att skapa en bra översikt och tydlighet.

En automatiserad arbetsprocess baserad på node.js ska finnas för webbapplikationen för att sammanställa, optimera och automatisk publicera. Bildmaterial som används ska även det optimeras. Källkoden ska version hanteras och JavaScript koden ska dokumenteras.

1.1 Målgruppsanalys

Webbapplikationens målgrupp är främst person bosatta i Stockholm 18–65 år som vill ha snabb och enkel information om badplatser i området. Barnfamiljer är också en del av målgruppen som önskar mer detaljerad information om badplatsen för att avgöra barnvänlighet och tillgänglighet samt planera ett besök i förväg. En relevant del av målgruppen är också tillfälliga besökare som vill hitta badplatser och behöver tydlig information i form av karta över vart badplatserna ligger.

2 Teori

2.1 Automatiserad arbetsprocess

En vanlig arbetsprocess inom webbutveckling är en automatiserad arbetsprocess där verktyg används för att automatisera uppgifter som annars skulle vara väldigt tidskrävande. Källkodfilerna, filer som jobbas med under utvecklig, omvandlas och optimeras samt omvandlar kod tex JavaScript till en version de flesta webbläsare stödjer för publicering för att fungera mer optimalt för webben [1].

Node.js möjliggör exekvering av JavaScript-kod utanför webbläsaren genom en öppenkällkodsbaserad miljö som gör att JavaScript går att köra på serversidan [2].

NPM står för **Node Package Manager** och används för att hantera och distribuera JavaScript-paket och bibliotek för node.js-miljön. Genom att använda NPM, som innehåller viktigt information, beroenden och andra konfigurationsdetaljer, i ett node.js-projekt möjliggörs enkel installation av verktyg och tredjepartsbibliotek som behövs [3].

Vite är ett snabbt och modernt byggverktyg som levererar snabba utvecklingsupplevelser för webbapplikationer [4]. Vite används främst med JavaScript-ramverk men fungerar även för webbprojekt utan ramverk som utvecklings- och byggverktyg. Vite bygger och optimerar kod för produktion för att smidigt kunna utveckla front-end-applikationer [5].

2.2 CSS Animationer

Med endast CSS går det att skapa rörelse och övergångar på en webbsida. Med CSS animeringar kan element förändra egenskaper såsom storlek, färg, position mm för att skapa mer levande interaktioner [6].

Keyframes används för att definiera stil på ett element vid specifika tidpunkter för att skapa en CSS animering. Keyframes använder sig av procentenheter (0–100%) eller from/to som selektorer för när något ska hända i animationen. För att skapa keyframes krävs namn på animationen, keyframes-selektorer och CSS-regler. Se figur 1 för kodexempel [7]. För att sedan använda keyframes används animationsattributet för att specificera vilken keyframes som ska användas (animation-name) samt egenskaper som hur länge animationen ska pågå (animation-duration), hur många gånger animationen ska upprepas (animations-iteration-count) med flera [8].

Transition i CSS skapar enklare animeringar med hjälp av mjuka övergångar mellan CSS-regler. Egenskaper för ett element, tex bredd, färg med flera, förändras då när man för muspekaren över elementet eller klickar på

det. Transition attribut styr hur övergångarna ska bete sig med bl.a. transition-duration som specificerar hur lång övergången ska vara, transition-property som anger vilken CSS-regel som ska ändra värde med flera [9].

2.3 CSS-preprocessorer

CSS-preprocessorer är en programvara med egen unik syntax för hur koden ska skrivas som används för att skriva CSS-kod på ett effektivare sätt som är lättare att underhålla. Det finns olika varianter av preprocessorer, några av de vanligaste är: SASS, LESS och Stylus. Gemensamt för de flesta preprocessorer är att de tillför mer funktionalitet som inte finns i vanlig CSS. För att en webbläsare ska kunna hantera en CSS-preprocessor måste en kompilator installeras som kompilerar preprocessor-koden till vanlig CSS som webbläsaren stöder och kan förstå [10].

SASS/SCSS är en CSS-preprocessor där SASS är originalsyntaxen som använder filändelsen .sass. SCSS som står för Sassy CSS är en nyare syntax inom SASS och har samma syntax som vanlig CSS-kod. SCSS använder filändelsen .scss [11].

Några funktionaliteter i SASS/SCSS:

Variables är ett sätt att lagra information som man vill återanvända. Färger och typsnitt är exempel på värden som kan lagras för att återanvändas [12].

Nesting tillåter att CSS-selektorer nästlas på ett sätt som följer HTML-kodens hierarki i ett projekt vilket organiserar koden bättre och kan underlätta läsbarheten [12].

Mixins grupperar CSS-egenskaper som sedan kan återanvändas vilket minskar redundans i koden. Det går att skicka med värden i en mixin för att göra den mer flexibel [12].

Partials gör det möjligt att dela upp koden i flera olika filer vilket kan underlätta underhållet av koden. Ett inledande understreck (_) i filnamnet indikerar att filen är en partial. För att en partial ska inkluderas för kompilering till CSS används regeln @use i huvudfilen [12].

2.4 JavaScript

Asynkron programmering tillåter JavaScript koden att köras asynkront istället för som standard köras uppifrån och ned. Detta gör att funktioner kan ”köras i bakgrunden” som till exempel ett externt anrop som tar tid innan det är klart utan att resten av JavaScript-koden kraschar eller fryser utan kan fortsätta köras [13].

AJAX står för Asynchronous JavaScript And XML och är en kombination av http-anrop till en webbserver för att hämta data och JavaScript och DOM-manipulation för att använda eller visa data. Ajax gör det möjligt att uppdatera delar av en webbsida utan att behöva ladda om sidan genom att uppdatera webbsidan asynkront genom att i bakgrunden anropa en webbserver för hämtning av data [14].

Promise är ett objekt i JavaScript som representerar ett asynkront värde som inte är tillgängligt direkt men förväntas bli tillgängligt senare. För att hantera resultatet av asynkrona funktioner används promises genom att arbeta med koden som körs när en asynkron funktion slutförts eller misslyckats [15].

Try/Catch är en syntax i JavaScript som tillåter att fel vid körning av koden ”fångas” och kan hanteras istället för att koden slutar köras och avbryts helt. Syntaxen består av två huvudblock, först Try och efter det Catch. Först körs koden i try och om inga fel uppstår så skippas Catch och koden körs vidare. Om ett fel uppstår i try så avslutas try-blocket och hoppar till Catch där en felkodsvariabel innehåller detaljer om felet [16].

Filter i JavaScript används för att filtrera en array utifrån satta villkor och skapa en ny array som endast innehåller element som uppfyller villkoren [17].

2.5 JSDoc

JSDoc är ett sätt att dokumentera och lägga till information till den kod som skrivits direkt i källkoden. Informationen om hur koden ska användas visas sedan som en informationsruta för att hjälpa andra utvecklare att använda och förstå koden. JSDoc kan även generera dokumentation utifrån den information och dokumentation som är dokumenterad i koden [18].

3 Metod

En planering för webbapplikationen ska skapas. Utifrån valda APIer ska funktioner för applikationen planeras.

Figma kommer användas i projektet för att skapa moodboard med färger och typsnitt samt för att skapa designskisser som visar webbapplikationens planerade utformning.

Vite kommer att användas som byggverktyg för att automatisera arbetsprocessen och optimera samt komprimera källkoden för publicering.

Källkoden kommer versions hanteras med Git/github och arbetsprocessen i Git kommer innehålla ett Main-repo för publicering och ett dev-repo för utveckling.

JSDoc ska användas för att dokumentera JavaScript kod.

För optimering av bilder kommer ett plugin till Vite med namnet Vite-plugin-image-optimizer användas, pluginet kommer hantera både raster- och vektorgrafik.

Ajax-anrop kommer användas för att uppdatera webbapplikationen med innehåll och filter-funktion i JavaScript för att filtrera resultat från AJAX-anrop utifrån, av användare angiven, sökning.

SASS/SCSS kommer användas för styling av webbapplikationen och för kompilering av SASS till CSS kommer npm-paketet SASS användas. Keyframes och transition kommer skrivas med SASS för att skapa animationer till webbapplikationen. För att anpassa färgschemat utifrån användaren val av mörkt eller ljust tema ska media query för detta användas.

4 Konstruktion

Initialt valdes vilka APIer som skulle användas för att skapa webbapplikationen och utifrån detta planerades också vilka funktioner som applikationen skulle ha, det landade i Stockholms Stads Öppna Datas webbtjänst Badplats (ur Hitta Service API) för information om badplatserna, Nominatim (OpenStreetMap) för GPS-koordinater för att skapa en ut markerad karta samt Meteoblue för väderprognos. Funktioner planerades och bestämdes för applikationen: innehåll från APIer ska hämtas med AJAX-anrop/fetchAPI-funktion, eventlyssnare för DOM-färdigladdat, eventlyssnare för input för sökfunktion, eventlyssnare för klick på unik badplats.

När initiala planeringen var satt så skapades en moodboard i Figma för att bestämma en grafik profil för webbapplikationen, vilka färger och typsnitt som ska användas och ikoner/bilder samt färgschemat för ljus/mörkt-tema. Se Bilaga A ”Moodboard” för bilder.

Utifrån moodboarden och initiala planeringen av funktioner skapades designskisser i Figma för webbapplikationen som visar den tänkta slutprodukten. Designskisserna inkluderar startsida med badplatser, inläsning av unik badplats samt layout för enheter med skärmar mindre än 750 px. Se Bilaga A ”Designskisser” för bilder.

Nu var grunden på plats och utvecklingen av webbapplikationen kunde starta. Ett Vite projekt startades upp med kommandot: npm create vite@latest i terminalen i projektmappen. Rensar projektet på vites automatgenererade innehåll och uppdaterar med egna JavaScript- och SASS-filer. Eftersom SASS används för styling behövs en kompilator, detta hämtas till projektet med npm-paketet SASS, kommando i terminalen npm install –save-dev sass.

Versionshantering startas med Git/github och dev-repo för utveckling startas. All programmering kommer att skrivas och testas i dev-branch, när allt är testat, klart och fungerande görs en merge med main-branchen som är kopplat för automatisk publicering och kommer då publiceras hos webhost.

En HTML-sida skapas med grunden för webbapplikationen i form av header, main, section och tomma div-element för att skriva ut innehåll till webbapplikationen som hämtas från APIer.

Skriver sedan en asynkron fetchAPI funktion (fetchData()) med try/Catch (se figur 1: kod fetchData-funktion) för att hämta data från Badplats APIet för att testa att anropet fungerar och kontrollera vilket data jag vill använda som innehåll. Lägger till en eventlyssnare som lyssnar efter DOM-färdigladdat, när färdigladdat körs funktionen fetchData. Från funktionen lagras den valda data från API hämtningen i en global-variabel (som senare ska användas till en sök-funktion) och data skickas vidare med funktionen writeData där html-element article skapas med valt innehåll från APIet som skrivs

ut till DOM. Detta blir då startsidans lista med Stockholms Stads alla badplatser.

```
async function fetchData() {
    try {
        const response = await fetch("https://apigw.stockholm.se/api/PublicHittaCMS/api/serviceunits?
&filter[servicetype.id]=104&page[limit]=1500&page[offset]=0&sort=name");
        const data = await response.json();

        allData = data.data;
        writeData(data.data);

    } catch (error) {
        console.error(`Felmeddelande ${error}`);
    }
}
```

Figur 1: Javascriptkod för AJAX-anrop med fetchAPI Catch/try.

Nu när det finns innehåll så skapas de SASS-partials som ska användas för styling. Grundstyling görs för webbapplikationen, variabler som lagrar färger, typsnitt och textstorlekar, mixin för article-elementet skapas och nesting används för stylingen. Strukturen för SASS-filer är en partial (base), en partial (layout) och main.scss för att hämta partials.

Lägger till ett inputfält på HTML-sidan och en eventlyssnare på fältet som lyssnar efter input. Vid input körs funktion searchFilter med den globala variabeln allData , en array som innehåller alla badplatser. SearchFilter använder sig av array filter funktionen i JavaScript som utefter input-värdet filtrerar arrayen och skapar en ny array som skickas med writeData som skriver ut sökresultatet till DOM.

Eventlyssnare för klick läggs till i writeData-funktionen som lyssnar efter klick på det skapade article-elementet. Vid klick körs asynkron funktion writeUniqueData, där URL för ny API-hämtning skickas med, som har en await som inväntar AJAX-anrop från fetchUniqeData. Asynkrona funktionen fetchUniqueData gör ett nytt AJAX-anrop med fetchAPI Catch/try från den medskickade url:en och returnerar sedan hämtade data till writeUniqeData som skriver ut nytt innehåll om unik badplats till DOM.

Ytterligare en asynkron funktion läggs till på eventlyssnaren för klick, showMap där gatuadress skickas med för att ett nytt AJAX-anrop behöver göras till Nominatim APIet för att från fram longitud och latitud för platsen. ShowMap-funktionen har en await på asynkrona funktionen fetchLocationData som returnerar latitud och longitud för den medskickade gatuadressen till showMap som skriver ut en karta med markör för platsen med hjälp av OpenStreetMap till DOM.

När webbapplikationen fått ytterligare innehåll återgår jag till styling och skapar en animation för att tydliggöra vilken badplats som är i fokus i listan av badplatser. Animationen skapas med hjälp av transition och transform på article-elementet där det zoomas in vid hover med scale.

Fortsätter sedan med styling av mörktema genom att lägga till en media query för detta. Se figur 2 för kodexempel media query. Detta gör nu att färgschemat ändras utefter användarinställning för ljust- eller mörkt tema.

```
@media (prefers-color-scheme: dark) {
  background-color: black;
}
```

Figur 2: Kodexempel för media query prefers-color-scheme

Lägger till bilden som ska användas som animation när webbapplikationen laddas in. Bilden glider över toppen på skärmen direkt när man kommer in på applikationen och förvinner succesivt så att fokus tas ifrån eventuell inladdningstid av badplatserna. För detta används keyframes som läggs i en egen SASS-partial med namnet keyframes. Animationen sker en gång när sidan laddas in.

Sista funktionen för webbapplikation som skrivs är för att hämta väder data från Meteoblue APIet. Skriver asynkron funktion för nytt AJAX-anrop med fetchAPT Try/Catch. Funktionen körs från showMap-funktionen som skickar med latitud och longitud. Data från API-hämtningen skickas sedan vidare till funktionen writeWeather som körs. Innehållet från API-data, datum, temperatur och ”pictocode” vilket anger ett väderpiktogram för dagen, skrivs ut till DOM. Från APIet dokumentation hämtas väderpiktogrammen och läggs till i projektet efter vilket pictocode de har. Vid utskrift till DOM angett pictocode vilket motsvarande bild som ska användas.

För att det grafiska materialet som nu finns i projektet ska optimeras för publicering hämtas plugin vite-image-optimizer samt tillägg för rester- och vektorgrafik. Inställningar för komprimering görs i vite-config-filen. Se figur 3 för vite-config-kod.

```
import { defineConfig } from "vite";
import { resolve } from "path";
import { ViteImageOptimizer } from "vite-plugin-image-optimizer";

export default defineConfig({
    build: {
        rollupOptions: {
            input: {
                main: resolve(__dirname, "index.html"),
            }
        }
    },
    css: {
        devSourcemap: true
    },
    plugins: [
        ViteImageOptimizer({
            png: {
                quality: 75
            },
            jpg: {
                quality: 75
            },
            jpeg: {
                quality: 75
            },
            webp: {
                quality: 70
            },
            avif: {
                quality: 60
            },
            svg: {
```

Figur 3: Vite-config-kod för komprimering av grafik.

Nu när alla funktioner var på plats hämtas JSDocs till projektet med kommandot npm install –save-dev jsdoc i terminalen och JSDoc skrivs för funktionerna i JavaScript och dokumentation för detta skapas med kommandot jsdoc [filnamn] som sparas i out-mappen i projektet. Se figur 4 för exempel på JSDoc-kommentar.

```
/**
 * Händelselyssnare för DOM färdigladdat
 * Anropar sedan fetchData för att hämta innehåll
 *
 * @event DOMContentLoaded
 * @returns {void} - returnerar inget värde utan anropar funktion när DOM laddat klart
 */
document.addEventListener("DOMContentLoaded", () => {
    fetchData();
```

Figur 4: JSdoc kommentar

Webbapplikationen testas genom att provköra alla funktioner, se att responsiviteten fungerar som tänkt, att den fungerar i webbläsare som Edge, Chrome och Firefox och att all kod validerar. Vid test upptäcks att det saknas ett sätt att ta sig tillbaka till listan med badplatser och sökfunktionen från den unika badplatsen. En tillbaka knapp läggs till i DOM och en eventlyssnare på button-elementet skapas för att lyssna efter klick och då köra funktionen writeData för att kunna återgå till badplats-listan och sökfunktionen. Designskiss uppdateras med knapp, se Bilaga A ”Ändring designskiss”. Även ny JSDoc dokumentation skrivs och skapas efter ändringen.

5 Resultat

Projektarbetet har resulterat i en ensides webbapplikation som visar Stockholms Stads badplatser och utökad information om respektive unik badplats inklusive väder och karta med markör för den aktuella platsen.

Innehållet i webbapplikationen uppdateras utan att sidan laddas om och det finns en sökfunktion där det går att filtrera urvalet genom input av fritext.

Animationer i form av en sol som glider över toppen på sidan när den öppnas och en zoom-effekt på article-elementen på respektive badplats i listan vid hover har implementerats.

Webbapplikationen har en genomgående grafisk profil och stylingen är gjorde med SASS/SCSS. Mixins, nesting och variabler har använts samt att partial-filer har skapats för olika delar av stylingen. Färgschemat för applikationen sätts utifrån användarens inställning för mörkt- eller ljust tema.

En automatiserad arbetsprocess är etablerad i projektet och i samband med byggandet sammanställs optimerade och komprimerade filer för publicering automatiskt och publiceras hos webhost. Källkoden har versionshanterats genom hela projektet och JavaScript koden är dokumenterad och dokumentation om koden är sammanställd i separat dokument.

6 Slutsatser

Målet med projektarbetet var att skapa en webbapplikation som visar Stockholms Stads badplatser inklusive information om badplatsen, väder och en karta genom att kombinera data från flera APIer. Webbapplikationen hämtar och presenterar de olika badplatserna, information om respektive badplats tillsammans med väderprognos och karta dynamiskt utan att sidan laddar om vilket visar att målet för detta projekt har uppnåtts.

Projektet har gett en djupare insikt och bättre förståelse för hur asynkron programmering fungerar och hur AJAX-anrop med Try/Catch är väldigt användbart för att applikationen inte ska dö och fel kan fångas upp på bra och tydligt sätt. Det har dock också varit en utmaning att få till alla olika fetchAPI-anrops som görs utan att något fastnat i promises på vägen.

Att skapa CSS-kod med SASS/SCSS har för mig bidragit till bättre struktur och gör koden lättare att underhålla och för ändringar att slå igenom större utan att man behöver ändra överallt. Animationer är en rolig del av CSS som verkligen skapar en känsla av att en webbplats ”lever”. Om jag har gjort om projektet igen hade jag förmodligen valt att lägga större vikt och mer tid för animationer för att webbapplikationen skulle kännas mer levande och mer modern än vad resultatet blev.

Utvecklingen av webbapplikation har känts smidig mycket tack vare den automatiserade arbetsprocessen som gör det både snabbare och enklare att testa, optimera och publicera en applikation. Att arbeta med detta har stärkt mina kunskaper om Git/Github, npm-paket och plugins, automatisk publicering med webhosts samt gett insikt i hur allt samverkar.

För att sammanfatta så har projektet gett mig bättre förståelse hur en modern dynamisk webbapplikation kan byggas upp, från planeringsstadiet till publiceringsstadiet.

Källförteckning

[1] MIUN Canvas, ”DT211G- Frontend-baserad webbutveckling/Arbetsprocessen, https://canvas.miun.se/courses/8221/pages/arbetsprocessen?module_item_id=195259, Hämtad, 2026-03-18, Publicerad: u. å

[2] Node.js, “Introduction to Node,js”, https://nodejs.org/en/learn/getting-started/introduction-to-nodejs, Hämtad: 2026-03-18, Publicerad: u. å

[3] MIUN Canvas, ”DT211G-Frontend-baserad webbutveckling/NodeJs och NPM, https://canvas.miun.se/courses/8221/pages/nodejs-och-npm?module_item_id=195258, Hämtad: 2026-03-18, Publicerad: u. å

[4] Vite, ”Getting Started”, https://vite.dev/guide/, Hämtad: 2026-03-18, Publicerad: u, å

[5] MIUN, ”DT211G-Frontend-baserad webbutveckling/Automatisering med Vite”, https://canvas.miun.se/courses/8221/pages/automatisering-med-vite?module_item_id=195261, Hämtad: 2026-03-18, Publicerad: u. å

[6] MIUN Canvas, ”DT211G-Frontend-baserad webbutveckling/Vad är en animering?”, https://canvas.miun.se/courses/8221/pages/vad-ar-en-animering?module_item_id=200021, Hämtad: 2026-03-18, Publicerad: u. å

[7] W3Schools, ”CSS @keyframes Rule”, https://www.w3schools.com/cssref/atrule_keyframes.php, Hämtad: 2026-03-18, Publicerad: u. å

[8] W3Schools, ”CSS Animation Properties”, https://www.w3schools.com/css/css3_animations_properties.asp, Hämtad: 2026-03-18, Publicerad: u. å

[9] MIUN Canvas, ”DT211G-Frontend-baserad webbutveckling/Transition”, https://canvas.miun.se/courses/8221/files/590722?module_item_id=201201&fd_cookie_set=1, Hämtad: 2026-03-08, Publicerad: u. å

[10] Mdn, ”CSS-Preprocessor”, https://developer.mozilla.org/en-US/docs/Glossary/CSS_preprocessor, Hämtad: 2026-03-18, Publicerad: 2026-01-25

[11] MIUN Canvas, ”DT211G-Frontend-baserad webbutveckling/Sass och Scss”, https://canvas.miun.se/courses/8221/pages/sass-och-scss?module_item_id=200012, Hämtad: 2026-03-18, Publicerad: u. å

[12] Sass-Lang, ”Sass Basics”, https://sass-lang.com/guide/, Hämtad: 2026-03-18, Publicerad: u. å

[13] W3Schools, ”Asynchronus Programming”, https://www.w3schools.com/JS//js_async.asp, Hämtad: 2026-03-18, Publicerad: u. å

[14] W3Schools, ”What is AJAX?”, https://www.w3schools.com/xml/ajax_intro.asp, Hämtad: 2026-03-19, Publicerad: u. å

[15] MIUN Canvas, ”DT211G-Frontend-baserad webbutveckling/Promises”, Hämtad: 2026-03-19, Publicerad: u. å

[16] Javascript.info, ”Error handling, ”try..catch”, https://javascript.info/try-catch, Hämtad: 2026-03-19, Publicerad: 2025-04-08

[17] Dahlgren M, Frontend-baserad webbutveckling - Föreläsning 2 [videofil]. [00:56:10] [citerad 2026-03-19]

[18] Medium, ”JSDocs: A quickstart Guide”, https://medium.com/@martink_rsa/js-docs-a-quickstart-guide-da6ce5df4a73, Hämtad: 2026-03-19, Publicerad: 2022-10-03

[19] Öppna Data, ”Dataportalen”, https://dataportalen.stockholm.se/dataportalen/, Hämtad: 2026-03-19, Publierad: u. å

[20] Nominatim, ”OpenStreetMap”, https://nominatim.openstreetmap.org/ui/search.html, Hämtad: 2026-03-19, Publicerad: u. å

[21] Meteoblue, https://www.meteoblue.com/, Hämtad: 2026-03-19, Publicerad: u. å

Bilaga A

Moodboard

Designskisser

Desktopskisser

Enheter under 750px

Ändring designskiss

Före ändring

Efter ändring